

Schritt 1: Technisches Setup in RStudio

Für unsere Analyse installieren wir das Paket `readxl`. Dieses Paket ermöglicht das Einlesen von Excel-Dateien (.xls und .xlsx) in R.

- Paket laden: Der erste technische Schritt im KDD-Prozess (Knowledge Discovery in Databases) ist der Import der Daten. Mit der folgenden Funktion laden wir die speziell für diese Aufgabe bereitgestellten Transaktionsdaten. Wir installieren zunächst einmal `install.packages("readxl")` über das Konsol.
- Laden der Bibliothek: Die Funktion `library()` lädt ein bereits installiertes Paket in die aktuelle R-Sitzung. Erst danach stehen dessen Funktionen – beispielsweise `read_excel()` – zur Verfügung.

Im Gegensatz dazu ist `split()` eine Funktion, die bereits fest in die Programmiersprache R integriert ist und kein extra Laden eines Pakets erfordert (Conversation History).

Um sicherzustellen, dass die in unserem Projekt verwendeten Pakete und deren exakte Versionen gespeichert und für die Zukunft festgehalten werden, installieren wir das Paket `renv` über die Console. Nach der Installation führen wir das Befehl `renv::init()` aus, um es in unserem Projekt integrieren.

Schritt 2: Datenimport

Im ersten Schritt des KDD-Prozesses werden die bereitgestellten Transaktionsdaten importiert.

- Die Funktion `read_excel()` liest die Excel-Datei ein und durch Zuweisung `meine_daten <-` werden die Daten dauerhaft in der Variable `meine_daten` gespeichert und können in den nächsten Analyseschritten weiterverwendet werden.
- Kommentare: Da unsere Datei in einem Unterordner namens „data“ liegt, nutzen wir einen relativen Pfad. Dabei schreiben wir den Ordernamen, gefolgt von einem Vorwärts-Schrägstrich (/) und dem Dateinamen in Anführungszeichen. `read_excel("data/Transaktionen_Apriori.xlsx")`

Wichtiger Hinweis: Wenn wir die Funktion `read_excel("data/Transaktionen_Apriori.xlsx")` ohne Zuweisung ausführen, wird das Ergebnis nur in der Konsole angezeigt und nicht für spätere Schritte (wie die Gruppierung) gespeichert (Conversation History). Das andere Problem ist, dass die Daten nicht dauerhaft gespeichert werden. Sobald die Ausgabe über den Bildschirm gelaufen ist, sind sie für R wieder „vergessen“. Deswegen führen wir mit Zuweisung `<-`, damit die Daten in unserem Environment-Fenster (oben rechts in RStudio) angezeigt zu bekommen.

```
meine_daten <- read_excel("data/Transaktionen_Apriori.xlsx")
```

Schritt 3: Vorbereitung der Daten (Transformation)

Da der Apriori-Algorithmus keine gewöhnliche Tabelle, sondern Transaktionen (Warenkörbe) erwartet, müssen die Daten zunächst transformiert werden.

- 3.1: Gruppierung der Daten (Transformation) Die Ausgangsdaten enthalten für jedes gekaufte Produkt eine eigene Zeile. Für die Warenkorbanalyse müssen jedoch alle Produkte, die zu derselben Bestellnummer gehören, zu einem gemeinsamen Warenkorb zusammengefasst werden. Hierfür wird die integrierte R-Funktion `split()` verwendet. Sie gruppiert die Produktgruppen anhand der Bestellnummer und erstellt daraus eine Liste von Warenkörben.

Erläuterung: produkte enthält alle Produktgruppen. bestellungen enthält die zugehörigen Bestellnummern. split() gruppiert sämtliche Produkte anhand ihrer Bestellnummer.

- Wir speichern die Bestandteile vorher in Variablen und dann gruppieren Produkte nach Bestellungen. Hier `trans_liste` ist eine Liste, aber noch kein Transaktionsobjekt.

```
produkte <- meine_daten$ProduktGroupName
bestellungen <- meine_daten$Bestellungsnummer

trans_liste <- split(produkte, bestellungen)
```

- 3.2 : Umwandlung in ein Transaktionsobjekt: Im nächsten Schritt wird die erzeugte Liste in ein Transaktionsobjekt umgewandelt. Hierzu wird das Paket `arules` verwendet. Dieses stellt die Daten in einem für den Apriori-Algorithmus geeigneten Format bereit, sodass die Transaktionen effizient verarbeitet und Assoziationsregeln ermittelt werden können.
- 3.2.2: wir installieren einmal `install.packages("arules")` über das Konsol
- 3.2.3: wir laden das Paket `library(arules)` in RStudio.
- 3.2.4: wir installieren einmal `install.packages("arulesViz")` über das Konsol für die Visualisierung der Regeln.
- 3.2.5: wir laden das Paket `library(arulesViz)` in RStudio.

```
library(arules)
```

```
## Lade nötiges Paket: Matrix
```

```
##
```

```
## Attache Paket: 'arules'
```

```
## Die folgenden Objekte sind maskiert von 'package:base':
```

```
##
```

```
##      abbreviate, write
```

```
library(arulesViz)
```

- 3.3: Transaktionsobjekt erzeugen: Das Paket `arules` erzeugt ein Transaktionsobjekt (`transactions`), das intern als spärliche Inzidenzmatrix (`sparse incidence matrix`) gespeichert wird. Dieses Format ermöglicht eine effiziente Verarbeitung durch den Apriori-Algorithmus.

```
warenkorb_trans <- as(trans_liste, "transactions")
```

- Kurze Überprüfung:

```
summary(warenkorb_trans)
```

```

## transactions as itemMatrix in sparse format with
## 10207 rows (elements/itemsets/transactions) and
## 38 columns (items) and a density of 0.08278374
##
## most frequent items:
##      Schutzblech      Mountainbike      Fahrradtasche      Schuhe E-Mountainbike
##           3805           2756           2375           2018           1940
##      (Other)
##           19215
##
## element (itemset/transaction) length distribution:
## sizes
##      1      2      3      4      5
## 800 2581 2668 2647 1511
##
##      Min. 1st Qu.  Median      Mean 3rd Qu.      Max.
## 1.000  2.000   3.000   3.146   4.000   5.000
##
## includes extended item information - examples:
##      labels
## 1 Accessoires
## 2      BMX
## 3      Brille
##
## includes extended transaction information - examples:
##      transactionID
## 1      41112
## 2      41113
## 3      41114

```

Schritt 4: Durchführung der Warenkorbanalyse

Nachdem die Daten erfolgreich in das Transaktionsformat überführt wurden, wenden wir nun den Apriori-Algorithmus an. Das Ziel ist es, logische Abhängigkeiten zwischen Produktgruppen zu erkennen, die häufig gemeinsam gekauft werden (Bundle-Angebote).

4.1: Begründung der gewählten Parameter

Für die Durchführung der Warenkorbanalyse wurden ein Mindestsupport von 5 % und eine Mindestkonfidenz von 30 % festgelegt.

- Mindestsupport (0,05): Eine Produktkombination muss in mindestens 5 % aller Transaktionen vorkommen, damit sie als relevant betrachtet wird. Dadurch werden sehr seltene Kombinationen ausgeschlossen, die für die Analyse meist wenig Aussagekraft besitzen.
- Mindestkonfidenz (0,30): Eine Regel wird nur berücksichtigt, wenn sie mit einer Wahrscheinlichkeit von mindestens 30 % eintritt. Das bedeutet beispielsweise: Wird Produkt A gekauft, soll in mindestens 30 % der Fälle auch Produkt B gemeinsam gekauft werden.

Die Werte wurden gewählt, um einen sinnvollen Kompromiss zwischen der Anzahl der gefundenen Regeln und deren Aussagekraft zu erreichen. Ein Mindestsupport von 5 % stellt sicher, dass nur häufig auftretende Produktkombinationen berücksichtigt werden. Die Mindestkonfidenz von 30 % filtert Regeln heraus, deren

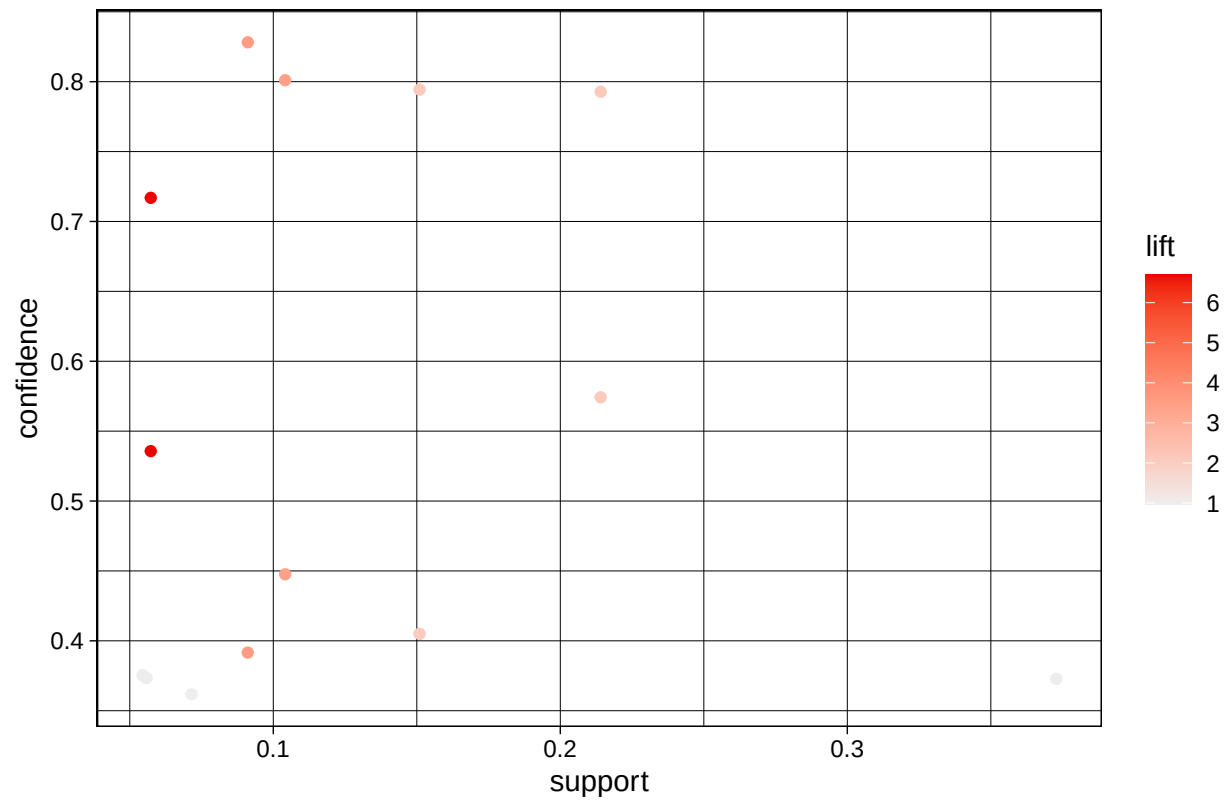
Vorhersagekraft zu gering ist. Dadurch bleiben ausreichend viele, aber dennoch aussagekräftige Assoziationsregeln für die weitere Analyse erhalten.

```
rules <- apriori(  
  warenkorb_trans,  
  parameter = list(  
    supp = 0.05,  
    conf = 0.30,  
    target = "rules"  
  )  
)
```

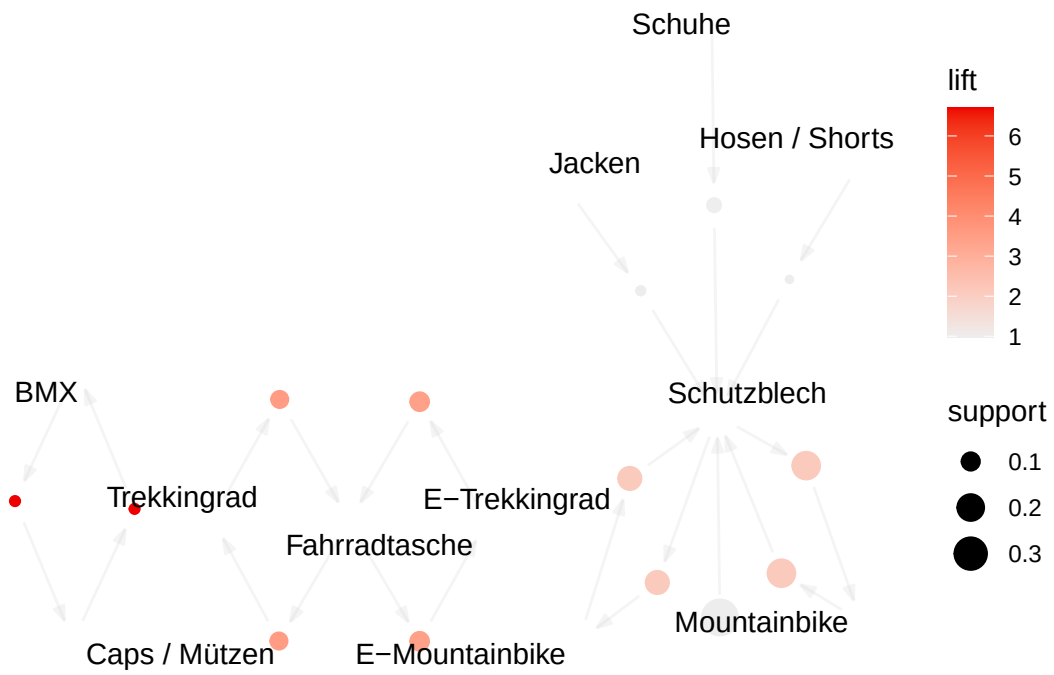
```
## Apriori  
##  
## Parameter specification:  
## confidence minval smax arem aval originalSupport maxtime support minlen  
##      0.3      0.1      1 none FALSE          TRUE      5      0.05      1  
## maxlen target  ext  
##      10  rules TRUE  
##  
## Algorithmic control:  
## filter tree heap memopt load sort verbose  
##    0.1 TRUE TRUE  FALSE TRUE     2     TRUE  
##  
## Absolute minimum support count: 510  
##  
## set item appearances ...[0 item(s)] done [0.00s].  
## set transactions ...[38 item(s), 10207 transaction(s)] done [0.00s].  
## sorting and recoding items ... [19 item(s)] done [0.00s].  
## creating transaction tree ... done [0.00s].  
## checking subsets of size 1 2 3 done [0.00s].  
## writing ... [14 rule(s)] done [0.00s].  
## creating S4 object ... done [0.00s].
```

```
plot(rules, method = "scatterplot")
```

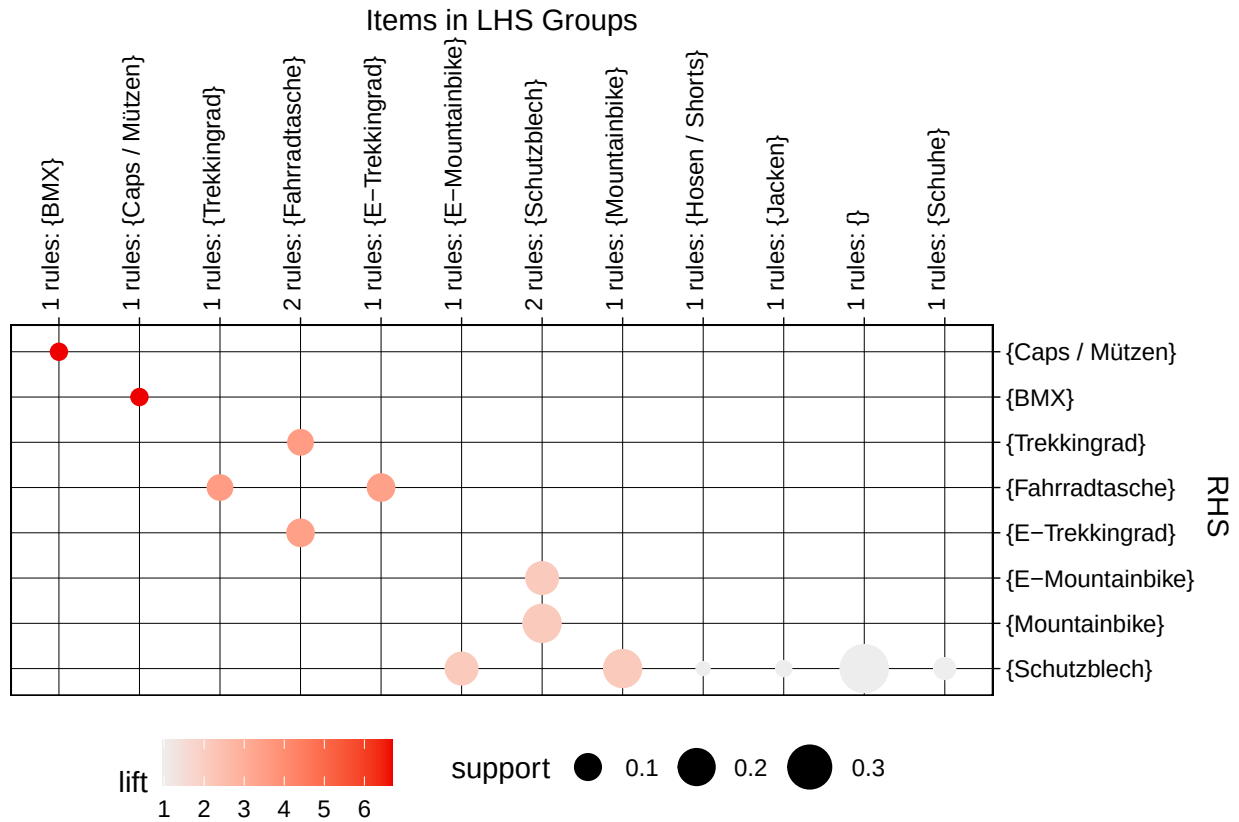
Scatter plot for 14 rules



```
plot(rules, method = "graph")
```

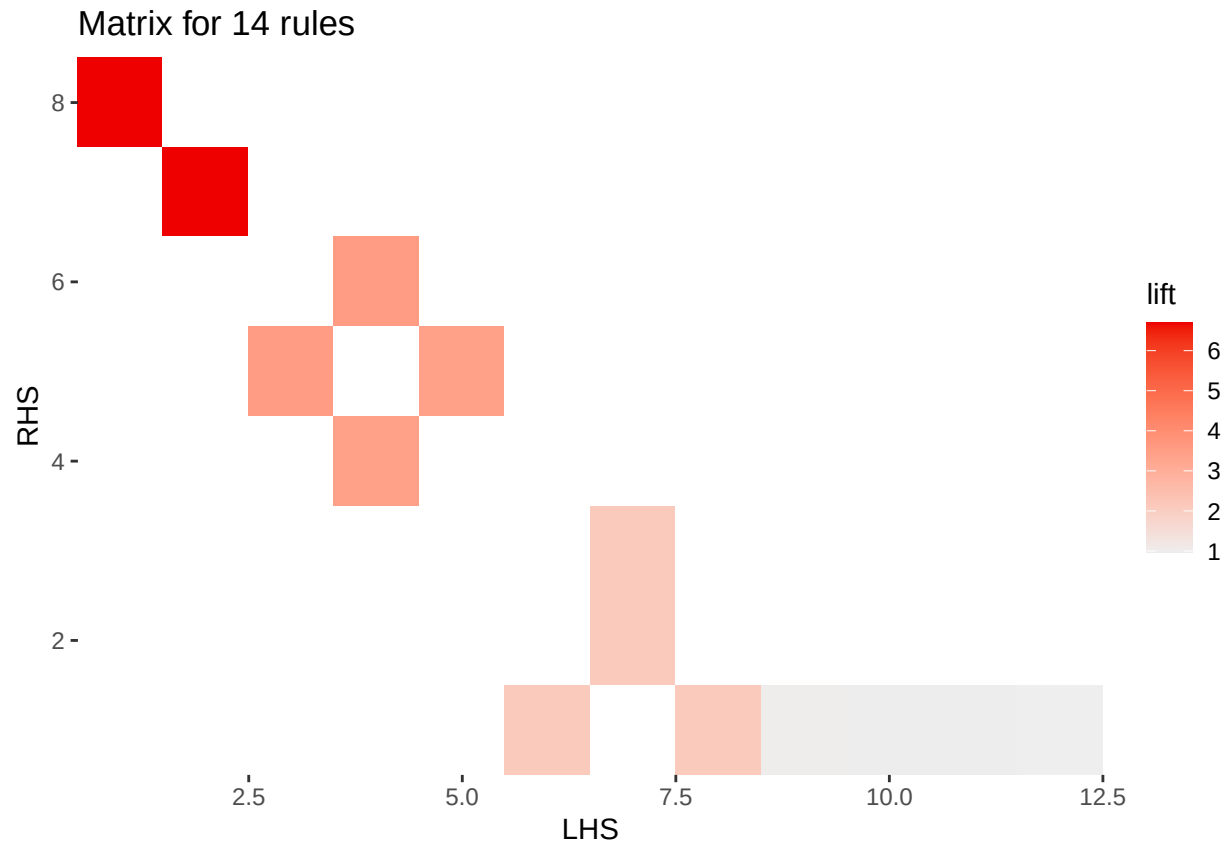


```
plot(rules, method = "grouped")
```



```
plot(rules, method = "matrix")
```

```
## Itemsets in Antecedent (LHS)
## [1] "{BMX}"           "{Caps / Mützen}" "{Trekkingrad}"   "{Fahrradtasche}"
## [5] "{E-Trekkingrad}" "{E-Mountainbike}" "{Schutzblech}"   "{Mountainbike}"
## [9] "{Hosen / Shorts}" "{Jacken}"         "{}"              "{Schuhe}"
## Itemsets in Consequent (RHS)
## [1] "{Schutzblech}"   "{Mountainbike}"   "{E-Mountainbike}" "{E-Trekkingrad}"
## [5] "{Fahrradtasche}" "{Trekkingrad}"    "{BMX}"            "{Caps / Mützen}"
```



- Regeln sortieren

```
rules_sorted <- sort(rules, by = "lift")
```

- Top-10-Regeln anzeigen

```
inspect(head(rules_sorted, 15))
```

##	lhs	rhs	support	confidence	coverage
## [1]	{BMX}	=> {Caps / Mützen}	0.05731361	0.7169118	0.07994514
## [2]	{Caps / Mützen}	=> {BMX}	0.05731361	0.5357143	0.10698540
## [3]	{Trekkingrad}	=> {Fahrradtasche}	0.09111394	0.8281389	0.11002253
## [4]	{Fahrradtasche}	=> {Trekkingrad}	0.09111394	0.3915789	0.23268345
## [5]	{E-Trekkingrad}	=> {Fahrradtasche}	0.10414421	0.8010550	0.13000882
## [6]	{Fahrradtasche}	=> {E-Trekkingrad}	0.10414421	0.4475789	0.23268345
## [7]	{E-Mountainbike}	=> {Schutzblech}	0.15097482	0.7943299	0.19006564
## [8]	{Schutzblech}	=> {E-Mountainbike}	0.15097482	0.4049934	0.37278338
## [9]	{Mountainbike}	=> {Schutzblech}	0.21406878	0.7928157	0.27001078
## [10]	{Schutzblech}	=> {Mountainbike}	0.21406878	0.5742444	0.37278338
## [11]	{Hosen / Shorts}	=> {Schutzblech}	0.05447242	0.3754220	0.14509650
## [12]	{Jacken}	=> {Schutzblech}	0.05584403	0.3732809	0.14960321
## [13]	{}	=> {Schutzblech}	0.37278338	0.3727834	1.00000000
## [14]	{Schuhe}	=> {Schutzblech}	0.07151955	0.3617443	0.19770746
##	lift	count			


```
## [1] 6.7010242 585
## [2] 6.7010242 585
## [3] 3.5590795 930
## [4] 3.5590795 930
## [5] 3.4426815 1063
## [6] 3.4426815 1063
## [7] 2.1308082 1541
## [8] 2.1308082 1541
## [9] 2.1267463 2185
## [10] 2.1267463 2185
## [11] 1.0070782 556
## [12] 1.0013347 570
## [13] 1.0000000 3805
## [14] 0.9703874 730
```